IIT
INSTITUT INTERNATIONAL TECHNOLOGIE
Université Nord Américaine privée

dqs ISO 21001: 2018 ASIIN Euro-Inf Bachelor awarded by EQANIE

TRAVAUX PRATIQUES 1

DM & MACHINE LEARNING

2ème année Génie Informatique SDIA

Semestre 1 ♣



M. Taoufik BEN ABDALLAH

taoufik.benabdallah@iit.ens.tn

M. Radhouane Guermazi

rguermazi@gmail.com

2026-2027

Correction TP1

1/ Charger le jeu de données **"athlete.csv"** dans un DataFrame nommé **df_athlete**, puis afficher sa taille et les types de données de chaque attribut

```
df_athlete=pd.read_csv("C:/…/athlete.csv")
print(df_athlete.shape) #(213, 4)
```

```
print(df_athlete.dtypes)
```

```
Gender          object
Age             object
Height/ Weight  object
Sport           object
dtype: object
```

	Gender	Age	Height/ Weight	Sport
0	M	24	180.0/80.0	Basketball
1	M	23	170.0/60.0	Judo
2	M	28	175.0/64.0	Gymnastics
3	M	28	175.0/64.0	Gymnastics
4	M	X	175.0/64.0	Basket
…	…	…	…	…
208	M	31	180.0/64.0	Athletics
209	M	X	178.0/81.0	Judo
210	M	33	182.0/77.0	Basket
211	M	X	193.0/78.0	Athletics
212	M	25	186.0/84.0	Athletics

213 rows × 4 columns

Correction TP1

2/ Diviser les valeurs de la colonne **`Height/Weight`** dans **`df_athlete`** en deux colonnes distinctes en utilisant le caractère **"/"** comme séparateur. Ajouter ensuite ces deux nouvelles colonnes, nommées **`Height`** et **`Weight`**, à **`df_athlete`** en les plaçant en troisième et quatrième position (**NB.** Ces colonnes remplaceront la colonne **`Height/Weight`** existante)

```
df_athlete[['Height','Weight']]=df_athlete['Height/ Weight'].str.split("/",
                                                    expand=True)
```

```
df_athlete=df_athlete.iloc[:, [0,1,-2,-1,3]]

#Ou bien
df_athlete=pd.concat([df_athlete.iloc[:, [0,1]],
                      df_athlete.iloc[:, [-2,-1]], df_athlete.iloc[:, 3]], axis=1)
```

	Gender	Age	Height	Weight	Sport
0	M	24	180.0	80.0	asketball
1	M	23	170.0	60.0	Judo
2	M	28	175.0	64.0	Gmnastics
3	M	28	175.0	64.0	Gmnastics
4	M	X	175.0	64.0	Basket
...	...	...	...	...	...

Correction TP1

3/ Remplacer les valeurs «Basket» par «Basketball» dans la colonne **Sport** de **df_athlete**. Afficher les 5 premières lignes de **df_athlete**

```
df_athlete["Sport"]=df_athlete["Sport"].replace({"Basket":"Basketball"})

#Ou bien
df_athlete["Sport"]=np.where(df_athlete["Sport"]=="Basket",
                             "Basketball", df_athlete["Sport"])

df_athlete.head()
```

	Gender	Age	Height	Weight	Sport
0	M	24	180.0	80.0	Basketball
1	M	23	170.0	60.0	Judo
2	M	28	175.0	64.0	Gymnastics
3	M	28	175.0	64.0	Gymnastics
4	M	X	175.0	64.0	Basket

	Gender	Age	Height	Weight	Sport
0	M	24	180.0	80.0	Basketball
1	M	23	170.0	60.0	Judo
2	M	28	175.0	64.0	Gymnastics
3	M	28	175.0	64.0	Gymnastics
4	M	X	175.0	64.0	Basketball

Correction TP1

4/ Renommer la colonne `"Gender"` en `"Sex"` dans **df_athlete**. Afficher les `3` *dernières* lignes de **df_athlete**

```
df_athlete.rename(columns={"Gender":"Sex"},inplace=True)

df_athlete.tail(3)
```

	Sex	Age	Height	Weight	Sport
210	M	33	182.0	77.0	Basket
211	M	X	193.0	78.0	Athletics
212	M	25	186.0	84.0	Athletics

5/ Afficher le sport *le plus fréquemment pratiqué* pour chaque sexe (`M` et `F`)

```
ct=pd.crosstab(df_athlete['Sex'], df_athlete['Sport'])

ct.idxmax(axis=1)
```

Sport	Athletics	Basketball	Football	Gymnastics	Judo
Sex					
F	24	1	3	16	1
M	89	15	23	36	5

```
Sex
F    Athletics
M    Athletics
dtype: object
```

Correction TP1

6/ Créer une série, nommé **S_HW**, en utilisant les valeurs de hauteur (**Height**) et de poids (**Weight**) selon la formule suivante : **Height-Weight-100**. Ensuite, ajouter une colonne nommée "State" en dernière position dans **df_athlete**. Cette colonne doit contenir 3 valeurs discrètes : «perfect» si la valeur correspondante dans **S_HW** est comprise entre 0 et 10 inclus, «Excessive» si la valeur est *supérieure* à 10, et «Insufficiency» si la valeur est *inférieure* à 0

```
df_athlete[['Height','Weight']]=df_athlete[['Height','Weight']].astype("float")
```

```
S_HW=df_athlete['Height']-df_athlete['Weight']-100

df_athlete['State']=S_HW.apply(lambda x: "Insufficiency"

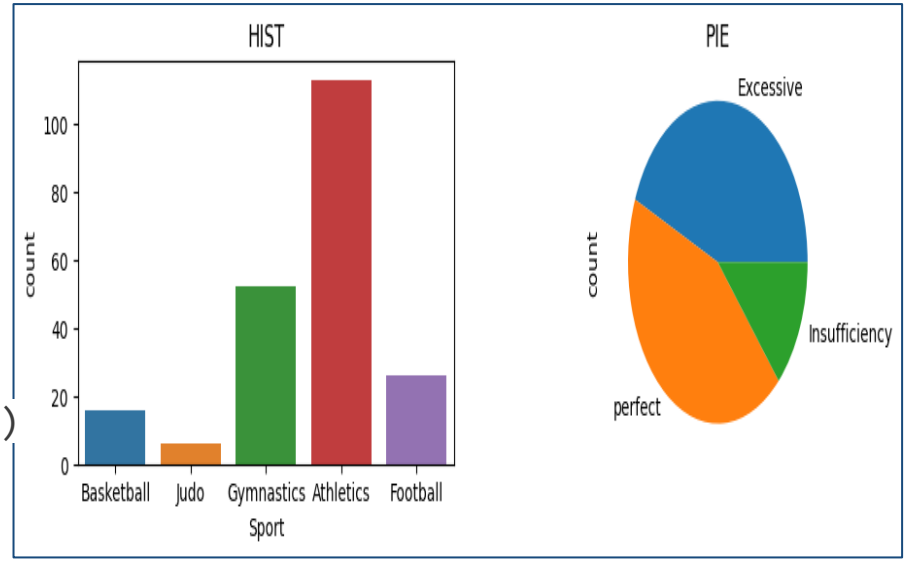
          if x<0 else("perfect" if x<10 else "Excessive"))
```

	Sex	Age	Height	Weight	Sport	State
0	M	24	180.0	80.0	Basketball	perfect
1	M	23	170.0	60.0	Judo	Excessive
2	M	28	175.0	64.0	Gymnastics	Excessive
3	M	28	175.0	64.0	Gymnastics	Excessive
4	M	X	175.0	64.0	Basket	Excessive
...	...	...	...	...	...	...

Correction TP1

7/ Créer une figure de taille `3×11` pouces comprenant deux axes alignés *horizontalement*. Le premier axe, intitulé «`HIST`», doit afficher un histogramme montrant la répartition des athlètes *selon leur sport*. Le deuxième axe, intitulé «`PIE`», doit afficher un diagramme en secteurs illustrant la distribution des athlètes *en fonction de leur état* (**`State`**)

```
plt.figure(figsize=(11, 3))
plt.subplot(1,2,1)
sns.countplot(x="Sport",data=df_athlete)
plt.title("HIST")
plt.subplot(1,2,2)
df_athlete['State'].value_counts().plot.pie()
plt.title("PIE")
plt.show()
```



Correction TP1

8/ Remplacer les valeurs « X » dans la colonne **Age** par la moyenne des âges des athlètes ayant le même sexe et le même sport que l'athlète concerné. Convertir la colonne **Age** de **df_athlete** en type *entier*. Afficher ensuite **df_athlete**

```
df=df_athlete[df_athlete['Age']!='X']
df['Age']= df['Age'].astype('int32')
ser=df.groupby(["Sex","Sport"])['Age'].mean()
df_athlete['Age']=df_athlet[['Age','Sex','Sport']].apply(lambda x:
                ser[x.iloc[1]][x.iloc[2]] if x.iloc[0]=='X'
                          else x.iloc[0], axis=1)

df_athlete['Age']= df_athlete['Age'].astype("int32")

df_athlete
```

	Sex	Age	Height	Weight	Sport	State
0	M	24	180.0	80.0	Basketball	perfect
1	M	23	170.0	60.0	Judo	Excessive
2	M	28	175.0	64.0	Gymnastics	Excessive
3	M	28	175.0	64.0	Gymnastics	Excessive
4	M	25	175.0	64.0	Basketball	Excessive
...	...	...	...	...	...	...

Correction TP1

9/ Trier les observations de **df_athlete** par *ordre décroissant* selon la colonne **Age** et réinitialiser les indices des lignes

```
df_athlete.sort_values(by="Age",
                       ascending=False, inplace=True)

df_athlete.reset_index(drop=True, inplace=True)
```

	Sex	Age	Height	Weight	Sport	State
0	M	37	174.0	63.0	Athletics	Excessive
1	F	35	155.0	43.0	Athletics	Excessive
2	M	35	178.0	61.0	Athletics	Excessive
3	M	34	178.0	66.0	Athletics	Excessive
4	M	34	190.0	125.0	Athletics	Insufficiency
...	...	...	...	...	...	...

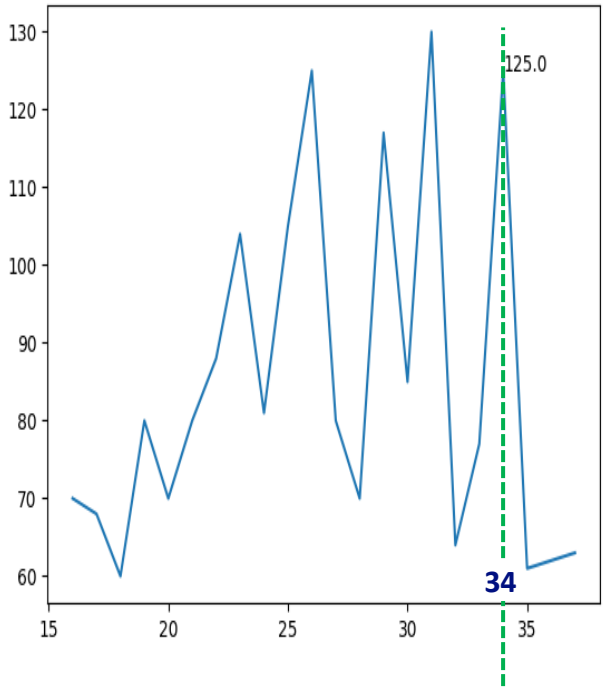
Correction TP1

10/ Écrire une fonction **`tracer_courbe(age=16)`** qui prend un paramètre **`age`** (par défaut `age=16`) et retourne un graphique montrant le ***poids maximum*** **des athlètes** en fonction de leur âge à partir de **df_athlete.** La fonction doit également **marquer la valeur du poids maximum** des athlètes pour l'âge spécifié (si cet âge existe dans les données).

Tester cette fonction avec `age=34`

```
def tracer_courbe(age=16):
    s=df_athlete.groupby(['Age'])['Weight'].max()
    plt.plot(s)
    if age in s.index :
        plt.annotate(s[age],xy=(age, s[age]),xytext=(age, s[age]))
    plt.show()

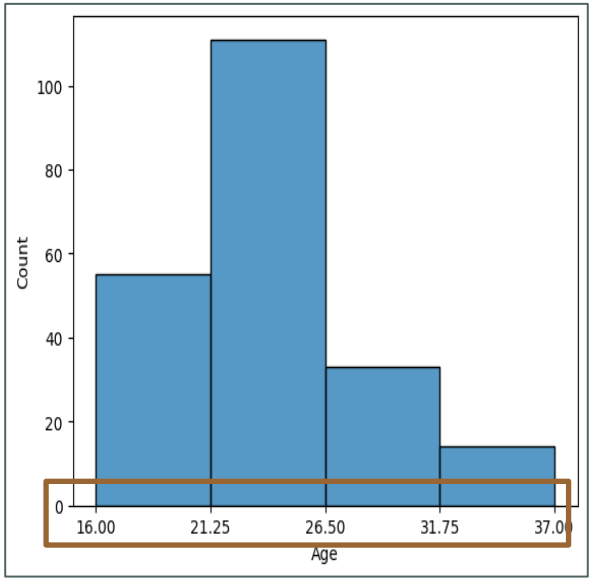

tracer_courbe(34)
```



Correction TP1

11/ Tracer un histogramme montrant la distribution des athlètes en fonction de `4` intervalles d'âge. Indiquer les valeurs des intervalles sur les axes des `x` (*`xticks`*) du graphique

```
b=np.linspace(df_athlete["Age"].min(), df_athlete["Age"].max(), 5)
plt.xticks(b)
sns.histplot(x="Age",data=df_athlete, bins=4)
plt.show()
```



Correction TP1

12/ Supprimer les athlètes dont l'âge appartient à l'intervalle d'âge *ayant le moins d'occurrences*

```
ser=df_athlete['Age'].apply(lambda x: 0
                    if x < b[1] else (1  if x < b[2]
                       else (2 if x < b[3] else 3)))
imin=ser.value_counts().idxmin()
df_athlete.drop(labels=list(ser[ser==imin].index),
                              axis=0, inplace=True)
df_athlete.reset_index(drop=True, inplace=True)
```

ser

0	3
1	3
2	3
3	3
4	3
..	..
208	0
209	0
210	0
211	0
212	0

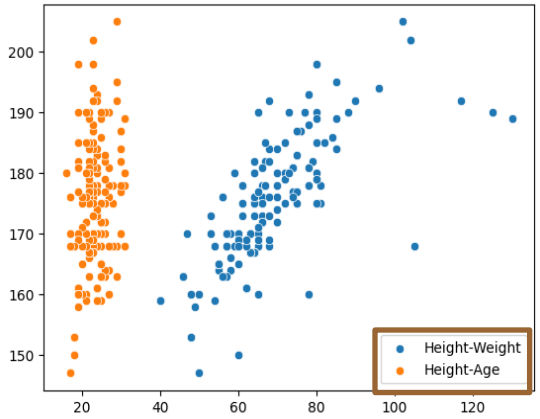
	Sex	Age	Height	Weight	Sport	State
0	M	31	178.0	61.0	Athletics	Excessive
1	M	31	180.0	64.0	Athletics	Excessive
2	M	31	189.0	130.0	Athletics	Insufficiency
3	F	31	168.0	60.0	Football	perfect
4	M	30	184.0	85.0	Athletics	Insufficiency
...	...	...	...	...	...	...
194	F	17	170.0	53.0	Athletics	Excessive
195	M	17	168.0	54.0	Athletics	Excessive
196	M	17	176.0	68.0	Athletics	perfect
197	F	17	147.0	50.0	Athletics	Insufficiency
198	M	16	180.0	70.0	Athletics	Excessive

199 rows × 6 columns

Correction TP1

13/ Tracer le nuage de points (*scatter*) de **`Height`** en fonction de **`Weight`** ainsi que de **`Height`** en fonction d'**`Age`**. Ajouter une *légende* à la figure et supprimer les titres des axis

```
sns.scatterplot(data=df_athlete,y="Height", x="Weight")
sns.scatterplot(data=df_athlete, y="Height", x="Age")
plt.legend(["Height-Weight", "Height-Age"])
plt.xlabel("")
plt.ylabel("")
plt.show()
```



14/ Transformer les colonnes **`Height`** et **`Weight`** de **`df_athlete`** en deux tableaux (*ndarray*): le premier, un tableau `1D` nommé **`X`**, pour la colonne **`Height`**, et le second, un tableau `2D` nommé **`Y`**, pour la colonne **`Weight`**

```
X=df_athlete.loc[:,'Height'].to_numpy()
print(X.shape) # (199,)
Y=df_athlete.loc[:,['Weight']].to_numpy()
print(Y.shape) # (199, 1)
```